CHMLTECH ONLINE JOURNAL · VOLUME 104, ISSUE 6 · RESEARCH ARTICLE

Designing Machine Learning Systems: From Problem Framing to Production Operations

Quak Foo Lee
Chief Technology Officer, Chmltech Ltd.
Division of Artificial Intelligence and Machine Learning · Burnaby, BC, Canada

Published November 9, 2026 19 min read

ABSTRACT

Machine learning models deliver value only as components of larger systems that collect data, compute features, serve predictions, monitor behavior and support change over time. Much of the cost and risk of these systems lies outside the model itself. This paper reviews the end-to-end design of production machine learning systems from the perspective of November 2026. It begins with problem framing, including criteria for deciding when machine learning should not be used, and with the requirements of reliability, scalability, maintainability and adaptability. It then examines data engineering, covering label quality, class imbalance and data leakage, followed by feature engineering and training–serving skew, the bias–variance trade-off and cross-validation for unbiased model selection. Evaluation is treated in detail, with equations for precision, recall, F1, ROC-AUC, expected calibration error and the Brier score, and with the relationship between offline evaluation and online controlled experiments. The paper reviews batch, online and edge deployment patterns, the detection of data and concept drift using divergence measures such as the population stability index, and the literature on hidden technical debt, the ML Test Score, software engineering for machine learning, deployment

challenges and MLOps principles. Model cards and datasheets are presented as documentation practices for responsible deployment. A design checklist consolidates the review. The paper concludes that production readiness requires evidence across every life-cycle stage rather than excellence in modeling alone.

Keywords: machine learning systems; MLOps; problem framing; model evaluation; calibration; concept drift; technical debt; model cards

1. Introduction

A machine learning (ML) model is a small part of a machine learning system. Sculley et al. observed that in real-world systems only a small fraction of the code is the model itself, and that the surrounding infrastructure for data collection, feature extraction, verification, serving, configuration and monitoring is larger, more complex and a principal source of long-term maintenance cost [1]. Huyen frames the design of such systems as an iterative process in which business objectives, data, models, deployment and monitoring are revisited repeatedly rather than completed in sequence [2]. Surveys of deployment case studies report that practitioners encounter difficulties at every stage of this process, from data management through model learning, verification and deployment, together with cross-cutting concerns of ethics, law, end-user trust and security [3].

This paper reviews the end-to-end design of ML systems from the perspective of November 2026. It is organized around the life cycle of a production system. Section 2 addresses problem framing and the decision of whether to use ML at all. Section 3 sets out system requirements. Section 4 covers data engineering, including labeling, class imbalance and leakage, and Section 5 covers feature engineering. Section 6 discusses training and model selection, Section 7 evaluation metrics, and Section 8 the relationship between offline evaluation and online experiments. Section 9 reviews deployment patterns and Section 10 monitoring and drift. Section 11 addresses MLOps and technical debt, Section 12 responsible documentation, and Section 13 consolidates the discussion in a design checklist. Section 14 concludes. Figure 1 arranges these stages as a loop rather than a sequence: monitoring and retraining feed back into data work, and an evaluation that fails to beat the baseline sends the project back to problem framing.

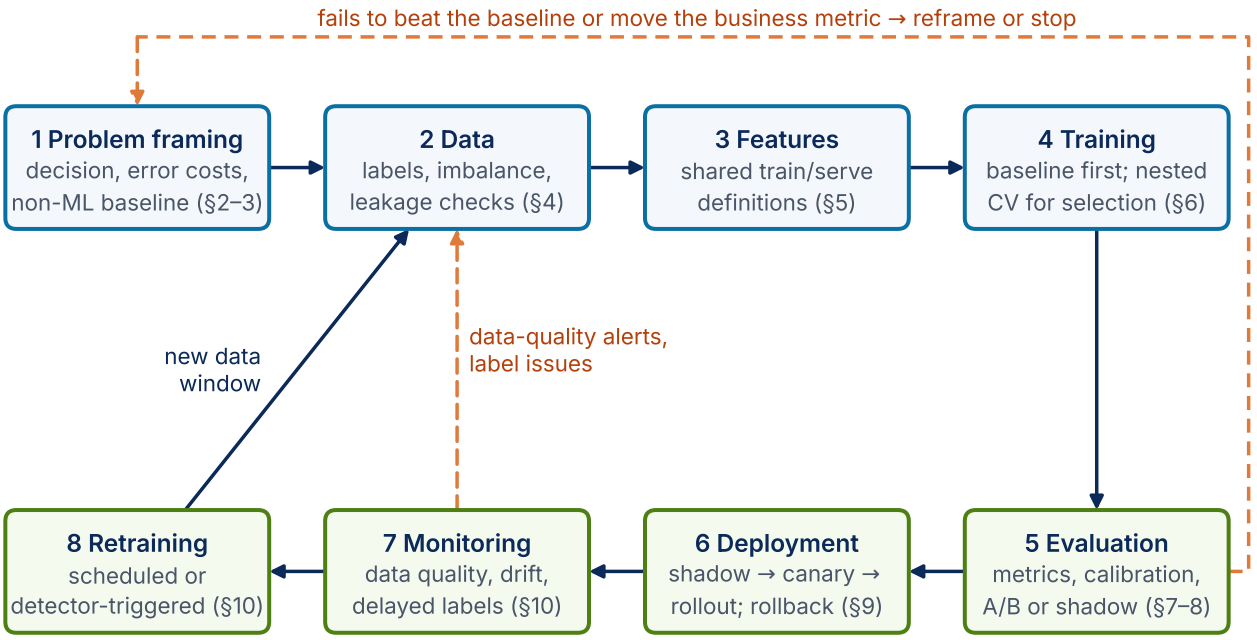


Figure 1. Life cycle of a production ML system. Development stages (blue) lead to production stages (green); solid arrows show the main loop, dashed arrows the feedback paths. Section numbers refer to this paper.

Drawn by the author from Sections 2–11, after [2] and [4].

2. Problem Framing and When Not to Use Machine Learning

Design begins with the decision that the system is meant to support, not with a model. A useful framing specifies who acts on the prediction, what action they take, how quickly the prediction is needed, what an error costs in each direction, and how success will be measured in business terms. Huyen distinguishes business objectives from ML objectives and emphasizes that model metrics are useful only insofar as they move the business metric that motivated the project [2]. The same problem can usually be framed in several ways, for example as classification, regression or ranking, and the framing determines which labels are needed, which loss is optimized and how the output is consumed downstream [2].

Machine learning is appropriate when a task involves patterns that are too complex or too changeable to specify as rules, when examples of the desired behavior exist or can be collected, when the task is repeated often enough for the investment to be recovered, and when occasional errors are tolerable or can be caught [2]. Conversely, ML should be avoided or deferred when a deterministic rule or a simple heuristic meets the requirement, when there

is no reliable signal or no way to obtain labels, when errors are unacceptable and cannot be detected, or when the organization cannot support the data pipelines and monitoring that a production model requires. Amershi et al. found at Microsoft that ML components differ from conventional software in the difficulty of data management, the specialized skills needed to customize and reuse models, and the way models become entangled with one another, so that ML imposes engineering costs beyond model training [4]. A non-ML baseline is therefore a design deliverable in its own right: it establishes the value that a model must exceed to justify its complexity.

> Before building a model, document the decision it supports, the cost of each type of error, the business metric it is meant to move, and a non-ML baseline. A model that cannot be shown to beat the baseline on the business metric should not proceed to production.

3. System Requirements

Huyen identifies four requirements that most production ML systems must meet: reliability, scalability, maintainability and adaptability [2]. They are summarized with their principal design implications in Table 1. Reliability is complicated by the fact that ML systems can fail silently: a model that returns well-formed but wrong predictions raises no exception, so correctness must be monitored statistically rather than inferred from uptime [2, 5]. Scalability concerns not only request volume but also the number of models, features and training runs the organization must manage. Maintainability depends on reproducible pipelines, versioned data and code, and clear ownership. Adaptability is the capacity to respond to shifting data distributions and business needs by retraining and redeploying quickly and safely.

Table 1. Requirements for production ML systems and their design implications.

Requirement	Meaning in an ML context	Design implications
Reliability	Correct outputs at the required level of performance, including under adverse conditions	Input validation; statistical output monitoring; fallbacks to a baseline or rule [2, 6]
Scalability	Growth in traffic, data volume, model count and team size	Stateless serving; distributed training; shared feature and metadata stores [2, 7]
Maintainability	Ability of different people to understand, reproduce and change the system	Versioned data, code and models; reproducible pipelines; documented ownership [7, 1]
Adaptability	Ability to respond to drift and changing requirements	Drift detection; automated retraining and evaluation; staged rollout [8, 7]

Source: requirement categories from [2]; implications compiled from the cited works.

4. Data Engineering

4.1 Data Quality and Labeling

Data work frequently determines the outcome of an ML project. Sambasivan et al. interviewed 53 practitioners working on high-stakes AI in India, East and West African countries and the United States and described data cascades, compounding events with negative downstream effects that originate in data issues; they reported a 92% prevalence of such cascades among their participants [9]. Polyzotis et al. survey the data life cycle of production ML, including data understanding, validation, cleaning and preparation, and describe the challenges that arise when these activities must operate continuously on changing production data rather than once on a static dataset [10]. A practical consequence is that data validation belongs in the pipeline itself: each new batch of training or serving data should be checked against an explicit schema covering expected features, types, value ranges and missing-value rates before it reaches a model, so that upstream changes are detected where they occur rather than inferred later from degraded predictions [10, 6].

Labels deserve particular scrutiny because they define what the model learns and how it is judged. Northcutt et al. estimated an average of at least 3.3% label errors in the test sets of ten widely used computer vision, language and audio datasets, including at least 6% of the ImageNet validation set, and showed that such errors can change which model appears best [11]. Confident learning provides a principled method for estimating the joint distribution of noisy and true labels and identifying likely mislabeled examples [12]. Practical labeling programs therefore define written labeling guidelines, measure inter-annotator agreement, audit a sample of labels, and version label sets so that changes in guidelines can be traced to changes in model behavior.

4.2 Class Imbalance

Many operational problems, such as fraud, equipment failure and defect detection, involve a rare positive class. He and Garcia review the effects of imbalance and the main remedies: sampling methods, cost-sensitive learning that assigns higher cost to minority-class errors, and evaluation metrics that remain informative when one class dominates [13]. The synthetic minority over-sampling technique (SMOTE) generates new minority examples by interpolating between a minority example and its nearest minority neighbors rather than duplicating examples [14]. Resampling changes the class prior seen during training, so predicted probabilities must be recalibrated (Section 7.3) before they are used as estimates of real-world risk, and resampling must be applied only to training folds, never to validation or test data.

4.3 Data Leakage

Kaufman et al. define leakage as the introduction of information about the target that should not legitimately be available to the model, and describe it as one of the most common mistakes in data mining practice, illustrated by controversies in several public competitions [15]. Typical sources include features computed after the prediction time, identifiers that encode the outcome, preprocessing statistics computed over the full dataset before splitting, and duplicate or near-duplicate records that appear in both training and test sets. Kaufman et al. propose a learn–predict separation in which each training example is constructed using only data available before its prediction point [15]. For temporal problems this implies time-based splits rather than random splits, and for grouped data, such as multiple records per customer or per machine, it implies splitting by group.

5. Feature Engineering

Feature engineering converts raw data into inputs that a model can use. Even where deep learning learns representations from raw inputs, production systems typically combine learned embeddings with engineered features such as aggregates over time windows, ratios, counts and domain-specific indicators [2]. Common operations include scaling numerical features, encoding categorical features (one-hot, target or hashed encodings), handling missing values explicitly, discretizing continuous variables and constructing interaction terms. Each transformation must be fitted on training data only and then applied unchanged at serving time; fitting a scaler or encoder on the full dataset is a form of leakage [15].

The principal systems risk in feature engineering is training–serving skew, in which features are computed differently in the offline training pipeline and the online serving path. Sculley et al. identify related forms of debt, including unstable and underutilized data dependencies, glue code and "pipeline jungles" of ad hoc preprocessing steps, and they note that because of entanglement, changing any input feature can change the behavior of the whole model, which they summarize as "changing anything changes everything" [1]. Shared feature definitions used by both training and serving, schema validation of inputs, and periodic pruning of features with little marginal value reduce these risks [1, 6].

6. Training and Model Selection

6.1 The Bias–Variance Trade-off

Model selection balances underfitting and overfitting. For squared-error loss, the expected prediction error of an estimator $\hat{f}$ at input x, with target $y = f(x) + \varepsilon$ and noise variance σ^2, decomposes into three terms [16, 17]:

$$\mathbb{E}[(y - \hat{f}(x))^2] = (\mathbb{E}[\hat{f}(x)] - f(x))^2 + \mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2] + \sigma^2 \tag{1}$$

The first term is the squared bias, the second the variance of the estimator over training sets, and the third the irreducible noise. Geman et al. showed that flexible models such as neural networks reduce bias at the cost of variance, so that controlling variance through regularization, more data or constrained model families is central to generalization [16]. In practice, a simple, well-regularized baseline such as logistic regression or a gradient-boosted tree model should be trained first; it calibrates expectations, exposes data problems early and provides the benchmark against which more complex models are judged [2].

6.2 Cross-Validation and Hyperparameter Selection

K-fold cross-validation estimates generalization error by training on *K* − 1 folds and evaluating on the held-out fold, rotating through all folds [17]. When the same cross-validation estimate is used both to tune hyperparameters and to report performance, the estimate is optimistically biased; Varma and Simon demonstrated this bias and showed that nested cross-validation, in which tuning is performed inside each outer training fold, yields an approximately unbiased estimate [18]. Splits must respect the structure of the data identified in Section 4.3: time-ordered or grouped splits for temporal or grouped data, and stratification by class for imbalanced problems. A final test set, held out from all development decisions, provides the last offline check before deployment.

7. Evaluation Metrics

7.1 Precision, Recall and F1

For a binary classifier with true positives TP, false positives FP and false negatives FN at a given decision threshold:

$$\text{Precision} = \mathrm{TP} / (\mathrm{TP} + \mathrm{FP}),\ \text{Recall} = \mathrm{TP} / (\mathrm{TP} + \mathrm{FN}),\ F_1 = 2 \cdot \text{Precision} \cdot \text{Recall} / (\mathrm{P}$$

Precision is the fraction of positive predictions that are correct, recall the fraction of actual positives that are found, and F_1 their harmonic mean. The appropriate balance depends on the error costs identified during problem framing (Section 2): a screening model that triggers a cheap manual review favors recall, while a model that triggers an expensive or irreversible action favors precision. Because both quantities depend on the threshold, the threshold should be chosen explicitly from the cost trade-off rather than left at a default of 0.5 [19].

7.2 ROC Analysis and Precision–Recall Curves

The receiver operating characteristic (ROC) curve plots the true positive rate against the false positive rate as the threshold varies. The area under the ROC curve (AUC) has a probabilistic interpretation: it equals the probability that the classifier assigns a higher score to a randomly chosen positive instance than to a randomly chosen negative instance [19]:

$$\mathrm{AUC} = P(s(x^{+}) > s(x^{-})) \tag{3}$$

where s is the classifier's score and x^{+} and x^{-} are positive and negative instances. AUC is threshold-independent and insensitive to class prevalence, which is useful for comparing rankers but can mask poor performance on a rare positive class. Saito and Rehmsmeier showed that precision–recall plots are more informative than ROC plots for evaluating binary classifiers on imbalanced datasets [20]. Table 2 summarizes these and related metrics. Figure 2 illustrates both points on a synthetic data set with 5% positives: ROC-AUC is 0.93, yet precision at the default threshold of 0.5 is only 0.37, and the threshold that minimizes a stated error cost differs from both 0.5 and the F_1-maximizing value.

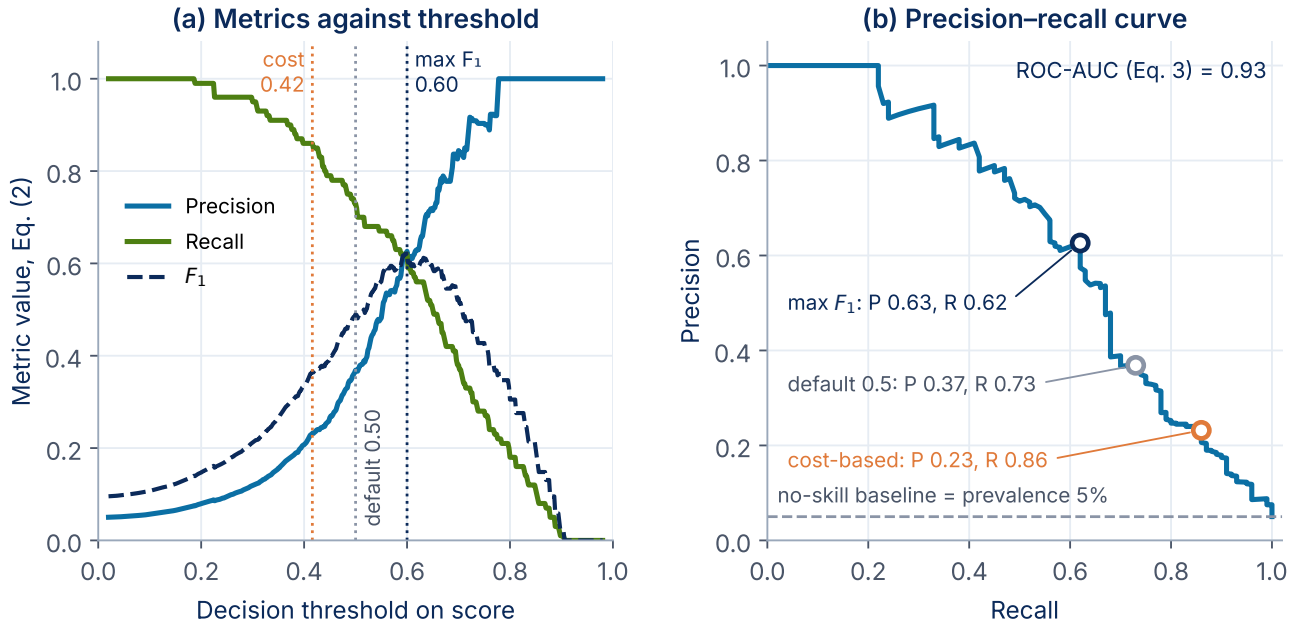


Figure 2. Threshold selection on synthetic data (2,000 cases, 5% positive). (a) Precision, recall and F_1 from Eq. (2) against the decision threshold, with the default of 0.5, the F_1-maximizing threshold and the threshold minimizing an assumed cost in which a missed positive costs 20 false alarms. (b) The precision–recall curve with the same three operating points and the prevalence baseline.

Author's calculation from Eqs. (2) and (3) on synthetic data.

7.3 Calibration

When predicted probabilities feed decisions, expected costs or downstream models, they must be calibrated, meaning that among instances assigned probability p, approximately a

fraction p are positive. Guo et al. found that modern neural networks are poorly calibrated, with depth, width, weight decay and batch normalization affecting calibration, and that temperature scaling, a single-parameter variant of Platt scaling, is effective on most datasets [21]. They measure miscalibration with the expected calibration error (ECE), which partitions n predictions into M confidence bins B_m and compares accuracy with mean confidence in each bin [21]. The Brier score, the mean squared difference between predicted probability p_i and outcome $o_i \in \{0, 1\}$, combines calibration and discrimination in a single proper scoring rule [22]:

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \left|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\right|, \quad \mathrm{BS} = \frac{1}{n} \sum_{i=1}^{n} (p_i - o_i)^2 \tag{4}$$

Table 2. Common evaluation metrics for classification systems and their appropriate use.

Metric	Measures	Use when	Caution
Precision, recall, F_1 (Eq. 2)	Error rates at one threshold	Error costs are asymmetric and the operating threshold is known	Threshold-dependent; F_1 weights precision and recall equally
ROC-AUC (Eq. 3)	Ranking quality across thresholds	Comparing rankers on moderately balanced data [19]	Can look favorable when positives are rare [20]
Precision–recall curve and its area	Precision attainable at each recall level	Positive class is rare [20]	Baseline depends on prevalence, so values are not comparable across datasets with different prevalence
ECE and Brier score (Eq. 4)	Agreement of probabilities with observed frequencies	Probabilities drive decisions or expected-cost calculations [21, 22]	ECE depends on the binning scheme
Business metric	Value delivered, such as cost avoided or conversions	Final go or no-go decisions [2]	Usually observable only online and with delay

Source: compiled from [19], [20], [21], [22] and [2].

8. Offline Versus Online Evaluation

Offline evaluation on historical data estimates how a model would have performed, but it cannot capture how users and processes react to the model's outputs, nor the feedback loops through which a deployed model alters its own future training data [1]. Online controlled experiments, or A/B tests, randomly assign units such as users or sessions to the existing system or the candidate and compare an overall evaluation criterion (OEC), a quantitative measure of the experiment's objective [23]. Kohavi et al. give a rule of thumb for the minimum sample size per variant at 95% confidence and 90% power:

$$n = (4r\sigma / \Delta)^2 \tag{5}$$

where r is the number of variants, σ is the standard deviation of the OEC and Δ is the minimum difference to be detected; the authors note that the factor of 4 may overestimate the requirement by about 25% for large n [23]. Because the required sample grows with the inverse square of Δ, detecting small improvements requires large samples or variance-reduction techniques. Kohavi, Tang and Xu discuss the practical conditions for trustworthy experiments, including guardrail metrics, checks that the observed allocation matches the intended allocation, and the dangers of stopping early when results look favorable [24]. Where randomized experiments are infeasible, as in many industrial process applications, shadow deployment, in which the candidate model scores live traffic without affecting decisions, and staged rollouts provide weaker but still valuable evidence [2].

9. Deployment Patterns

Deployment patterns differ mainly in when predictions are computed and where. In batch prediction, the model scores a population on a schedule and writes predictions to a store from which applications read them; this is simple and efficient but cannot react to information that arrives between runs. In online prediction, the model is served behind an interface and scores each request as it arrives, which supports fresh features and personalized responses but imposes latency, availability and scaling requirements [2]. Streaming designs compute features from event streams so that online predictions can use near-real-time information. Edge deployment runs the model on a device or local controller, reducing latency, network dependence and data transfer, but constraining model size and complicating updates and monitoring [2].

Paleyes et al. group deployment-stage challenges into integration, monitoring and updating, and note that integration difficulties often stem from infrastructure, code quality and the need to coordinate ML and software teams [3]. Whatever the pattern, releases should be progressive: shadow mode first, then a canary on a small share of traffic, then wider rollout, each stage gated by predefined metric thresholds and each with a tested rollback path. The ML Test Score rubric of Breck et al. includes corresponding infrastructure tests: model quality is validated before serving, models are canaried before serving, and serving models can be rolled back; its monitoring tests include checks that training and serving are not skewed and that models are not too stale [6].

10. Monitoring and Drift

10.1 Types of Drift

A model trained on one data distribution is deployed into a world that changes. Gama et al. define concept drift as a change in the joint distribution of inputs and targets over time, and distinguish real drift, a change in the conditional distribution $p(y \mid x)$, from virtual drift, a change in the input distribution $p(x)$ that does not alter the decision boundary; they further characterize drift by its pattern as sudden, incremental, gradual or recurring [8]. Lu et al. review drift detection, understanding and adaptation, and organize detection methods into error-rate-based, data-distribution-based and multiple-hypothesis-test approaches [25]. Rabanser et al. emphasize that ML systems tend to fail silently when inputs shift; in their experiments, two-sample testing on representations produced by a pretrained classifier was the most effective of the shift-detection methods they examined [5].

10.2 Distribution Monitoring with KL Divergence and PSI

Ground-truth labels often arrive late or never, so monitoring relies heavily on input and prediction distributions. The Kullback–Leibler divergence of a current distribution Q from a reference distribution P over bins i is defined as [26]:

$$D_{\mathrm{KL}}(P \parallel Q) = \Sigma_i \, p_i \ln(p_i / q_i), \; \mathrm{PSI} = \Sigma_i \, (a_i - e_i) \ln(a_i / e_i) \tag{6}$$

The population stability index (PSI), widely used in credit-risk model validation, compares the actual proportion a_i of a variable in bin i with the expected proportion e_i from the development sample. Algebraically, PSI equals the sum of the two directed divergences, $D_{\mathrm{KL}}(A \parallel E) + D_{\mathrm{KL}}(E \parallel A)$, so it is symmetric in the two distributions. Yurdakul and Naranjo note that the conventional rule-of-thumb thresholds of 0.10 and 0.25 are applied without reference to type I or type II error rates, and they study the statistic's sampling properties to support more principled use [27]. Both measures require care with empty bins, which make the logarithm undefined, and with the choice of binning. A monitoring design should combine such distribution statistics with data-quality checks (schema, missing rates, ranges), prediction-distribution monitoring, and, when labels become available, delayed performance metrics; alerts should be tied to retraining or investigation procedures rather than left as dashboards [6, 25]. Figure 3 shows on synthetic data why the threshold should be derived rather than

assumed: with 2,000 observations per check, a threshold set from the no-drift sampling distribution of PSI flags a gradual shift at week 6, whereas the 0.10 and 0.25 rules of thumb are first exceeded at weeks 11 and 15.

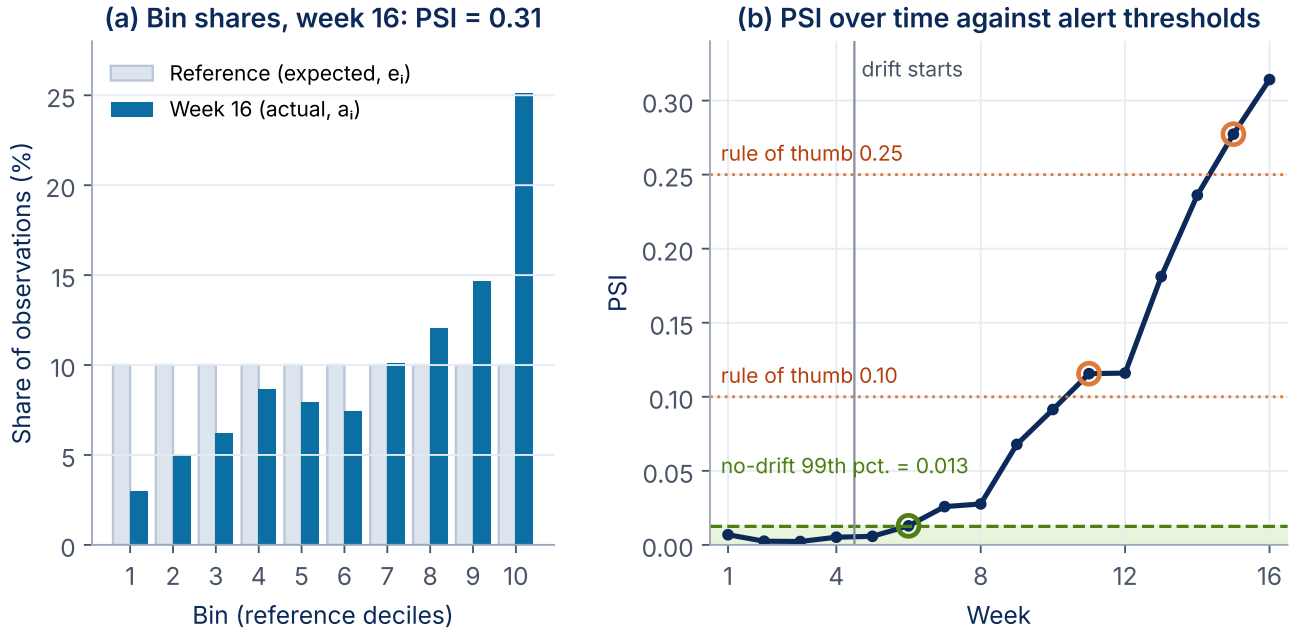


Figure 3. PSI monitoring on synthetic data: a standard normal input whose mean drifts by 0.05 standard deviations per week from week 5, with 2,000 observations per week and ten bins at the reference deciles. (a) Bin shares at week 16. (b) Weekly PSI from Eq. (6) against the 99th percentile of PSI without drift and the conventional 0.10 and 0.25 values. Alert thresholds are context-dependent: the no-drift percentile changes with sample size and binning and fixes a 1% false-alarm rate per check, while the rules of thumb have no stated error rates [27].

Author's calculation from Eq. (6) on synthetic data.

Once drift is detected, the system must adapt. Gama et al. distinguish informed adaptation, in which retraining is triggered by a detector, from blind adaptation, in which the model is updated at regular intervals regardless of evidence of change, and they review memory and forgetting mechanisms such as sliding windows and instance weighting that determine which historical data remain in the training set [8]. Lu et al. add drift understanding, establishing when, how and where a drift occurred, as a step between detection and adaptation, because the appropriate response to a sudden, localized change differs from the response to a gradual, global one [25]. In production, scheduled retraining provides a simple form of blind adaptation, while detector-triggered retraining reduces unnecessary updates; in either case, each retrained model should pass the same automated evaluation gates as the original before it replaces the serving model [7, 6].

Distribution drift is a signal for investigation, not proof of harm. Pair drift statistics with delayed ground-truth performance where possible, and define in advance which thresholds trigger investigation, retraining or rollback.

11. MLOps and Technical Debt

11.1 Hidden Technical Debt

Sculley et al. argued that ML systems incur technical debt that is hidden at the system level rather than visible in code. Their catalog includes the erosion of abstraction boundaries through entanglement, correction cascades in which models are stacked to patch one another, undeclared consumers of model outputs, unstable and underutilized data dependencies, direct and hidden feedback loops, system-level anti-patterns such as glue code, pipeline jungles and dead experimental code paths, configuration debt, and changes in the external world [1]. Shankar et al., interviewing 18 ML engineers across applications including chatbots, autonomous vehicles and finance, identified velocity, validation and versioning as the three variables governing success in production deployments [28].

11.2 The ML Test Score

Breck et al. turned production-readiness practice at Google into a rubric of 28 tests in four sections of seven: tests for features and data, tests for model development, tests for ML infrastructure, and monitoring tests. A test earns half a point if performed manually with documented results and one point if automated and run repeatedly; the final ML Test Score is the minimum of the four section scores, so that progress is required in every area [6]. The minimum aggregation is a useful design principle in its own right: an excellent model with no monitoring is not production-ready.

11.3 MLOps Principles

Kreuzberger et al. combined a literature review, a tool review and eight expert interviews to define MLOps as a paradigm, spanning best practices, concepts and a development culture, for the end-to-end conceptualization, implementation, monitoring, deployment and scalability of ML products, drawing on ML, software engineering (especially DevOps) and data engineering [7]. They identify nine principles: CI/CD automation, workflow orchestration,

reproducibility, versioning of data, model and code, collaboration, continuous ML training and evaluation, ML metadata tracking and logging, continuous monitoring, and feedback loops [7]. Amershi et al. describe a nine-stage ML workflow integrated into agile software processes at Microsoft, from model requirements and data collection through training, evaluation, deployment and monitoring, and stress that the stages contain feedback loops rather than a linear sequence [4]. The level of automation should be matched to the system's change rate and risk; a model retrained annually may not need continuous training, but every production model needs versioning, reproducibility and monitoring.

12. Responsible ML Documentation

Documentation makes a model's intended use, limitations and evidence available to the people who deploy, operate, audit and are affected by it. Model cards are short documents accompanying trained models that report benchmarked evaluation across relevant conditions and groups, including cultural, demographic, phenotypic and intersectional groups where these are relevant to the application, and that disclose the intended context of use and the evaluation procedures [29]. The proposed sections cover model details, intended use, factors, metrics, evaluation data, training data, quantitative analyses, ethical considerations, and caveats and recommendations [29].

Datasheets for datasets apply the same principle to data. Gebru et al. propose that every dataset be accompanied by a datasheet that documents its motivation, composition, collection process, preprocessing, cleaning and labeling, recommended uses, distribution and maintenance [30]. In industrial and enterprise settings, the same documents serve practical purposes beyond fairness: they record the operating envelope within which a model was validated, the data sources on which it depends, and the conditions under which it should not be used, which are exactly the facts needed during incident investigation and model handover.

13. Design Checklist

Table 3 consolidates the preceding sections into a design checklist organized by life-cycle stage. As with the ML Test Score, the value of the checklist lies in requiring evidence for every stage rather than excellence in one [6].

Table 3. Design checklist for production machine learning systems.

Stage	Key questions	Evidence required
Problem framing	What decision is supported? What do errors cost? Is ML needed?	Decision and cost statement; business metric; non-ML baseline [2, 4]
Requirements	What reliability, scalability, maintainability and adaptability are required?	Service-level targets; fallback behavior; retraining cadence [2]
Data	Are labels accurate? Is the class balance understood? Is leakage excluded?	Label audit and agreement; time- or group-based splits; datasheet [11, 15, 30]
Features	Are transformations identical in training and serving?	Shared feature definitions; schema validation; skew tests [1, 6]
Training and selection	Is the estimate of generalization unbiased?	Nested or held-out evaluation; simple baseline comparison [18, 17]
Evaluation	Do metrics reflect error costs, imbalance and calibration needs?	Precision–recall at the chosen threshold; calibration metrics; subgroup results [20, 21, 29]
Online validation	Does the model improve the business metric in practice?	A/B test with adequate power or shadow and canary evidence [23, 24]
Deployment	Batch, online or edge? How is rollback performed?	Progressive rollout plan; tested rollback [2, 3]
Monitoring	How are drift and silent failures detected?	Data-quality checks; drift statistics; delayed performance tracking; alert procedures [8, 5, 27]
Operations and governance	Is the system reproducible, versioned and documented?	ML Test Score self-assessment; MLOps principles; model card [6, 7, 29]

Source: synthesis of Sections 2–12; see the cited works for each item.

14. Conclusion

Designing a machine learning system is a systems-engineering task in which the model is one component among data pipelines, feature computation, serving infrastructure, monitoring and governance [1, 2]. The literature reviewed here converges on several principles. Problems should be framed around decisions and error costs, with a non-ML baseline as the benchmark. Data quality, label accuracy and the exclusion of leakage determine the ceiling on model performance [9, 11, 15]. Evaluation should use metrics matched to class balance, error costs and the need for calibrated probabilities, and should be completed by online evidence where possible [20, 21, 24]. Deployed models must be monitored for drift and silent failure [8, 5], and the system as a whole should be assessed for production readiness across data, model, infrastructure and monitoring, with documentation that records its intended use and limits [6, 7, 29, 30]. Systems designed on these principles can be maintained, audited and improved as their data and requirements change.

References

1. Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden technical debt in machine learning systems. Advances in Neural Information Processing Systems, 28, 2503–2511. proceedings.neurips.cc/paper/2015/hash/86df7dcfd896fcaf2674f757a2463eba-Abstract
2. Huyen, C. (2022). Designing machine learning systems: An iterative process for production-ready applications. O'Reilly Media. ISBN 978-1-098-10796-3. www.oreilly.com/library/view/designing-machine-learning/9781098107956/
3. Paleyes, A., Urma, R.-G., & Lawrence, N. D. (2022). Challenges in deploying machine learning: A survey of case studies. ACM Computing Surveys, 55(6), 1–29. 10.1145/3533378
4. Amershi, S., Begel, A., Bird, C., DeLine, R., Gall, H., Kamar, E., Nagappan, N., Nushi, B., & Zimmermann, T. (2019). Software engineering for machine learning: A case study. 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), 291–300. 10.1109/ICSE-SEIP.2019.00042
5. Rabanser, S., Günnemann, S., & Lipton, Z. C. (2019). Failing loudly: An empirical study of methods for detecting dataset shift. Advances in Neural Information Processing Systems 32 (NeurIPS 2019). arXiv:1810.11953. arxiv.org/abs/1810.11953

6. Breck, E., Cai, S., Nielsen, E., Salib, M., & Sculley, D. (2017). The ML test score: A rubric for ML production readiness and technical debt reduction. 2017 IEEE International Conference on Big Data (Big Data), 1123–1132. 10.1109/BigData.2017.8258038
7. Kreuzberger, D., Kühl, N., & Hirschl, S. (2023). Machine learning operations (MLOps): Overview, definition, and architecture. IEEE Access, 11, 31866–31879. 10.1109/ACCESS.2023.3262138
8. Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. ACM Computing Surveys, 46(4), 1–37. 10.1145/2523813
9. Sambasivan, N., Kapania, S., Highfill, H., Akrong, D., Paritosh, P., & Aroyo, L. M. (2021). "Everyone wants to do the model work, not the data work": Data cascades in high-stakes AI. Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems, 1–15. 10.1145/3411764.3445518
10. Polyzotis, N., Roy, S., Whang, S. E., & Zinkevich, M. (2018). Data lifecycle challenges in production machine learning: A survey. ACM SIGMOD Record, 47(2), 17–28. 10.1145/3299887.3299891
11. Northcutt, C. G., Athalye, A., & Mueller, J. (2021). Pervasive label errors in test sets destabilize machine learning benchmarks. 35th Conference on Neural Information Processing Systems (NeurIPS 2021) Track on Datasets and Benchmarks. arXiv:2103.14749. arxiv.org/abs/2103.14749
12. Northcutt, C., Jiang, L., & Chuang, I. (2021). Confident learning: Estimating uncertainty in dataset labels. Journal of Artificial Intelligence Research, 70, 1373–1411. 10.1613/jair.1.12125
13. He, H., & Garcia, E. A. (2009). Learning from imbalanced data. IEEE Transactions on Knowledge and Data Engineering, 21(9), 1263–1284. 10.1109/TKDE.2008.239
14. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. Journal of Artificial Intelligence Research, 16, 321–357. 10.1613/jair.953
15. Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. ACM Transactions on Knowledge Discovery from Data, 6(4), 1–21. 10.1145/2382577.2382579
16. Geman, S., Bienenstock, E., & Doursat, R. (1992). Neural networks and the bias/variance dilemma. Neural Computation, 4(1), 1–58. 10.1162/neco.1992.4.1.1
17. Hastie, T., Tibshirani, R., & Friedman, J. (2009). The elements of statistical learning: Data mining, inference, and prediction (2nd ed.). Springer. 10.1007/978-0-387-84858-7
18. Varma, S., & Simon, R. (2006). Bias in error estimation when using cross-validation for model selection. BMC Bioinformatics, 7, 91. 10.1186/1471-2105-7-91

19. Fawcett, T. (2006). An introduction to ROC analysis. Pattern Recognition Letters, 27(8), 861–874. 10.1016/j.patrec.2005.10.010

20. Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. PLOS ONE, 10(3), e0118432. 10.1371/journal.pone.0118432

21. Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. Proceedings of the 34th International Conference on Machine Learning (ICML 2017). arXiv:1706.04599. arxiv.org/abs/1706.04599

22. Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. Monthly Weather Review, 78(1), 1–3. 10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2

23. Kohavi, R., Henne, R. M., & Sommerfield, D. (2007). Practical guide to controlled experiments on the web: Listen to your customers not to the HiPPO. Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 959–967. 10.1145/1281192.1281295

24. Kohavi, R., Tang, D., & Xu, Y. (2020). Trustworthy online controlled experiments: A practical guide to A/B testing. Cambridge University Press. 10.1017/9781108653985

25. Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., & Zhang, G. (2019). Learning under concept drift: A review. IEEE Transactions on Knowledge and Data Engineering, 31(12), 2346–2363. 10.1109/TKDE.2018.2876857

26. Kullback, S., & Leibler, R. A. (1951). On information and sufficiency. The Annals of Mathematical Statistics, 22(1), 79–86. 10.1214/aoms/1177729694

27. Yurdakul, B., & Naranjo, J. (2020). Statistical properties of the population stability index. The Journal of Risk Model Validation. 10.21314/JRMV.2020.227

28. Shankar, S., Garcia, R., Hellerstein, J. M., & Parameswaran, A. G. (2022). Operationalizing machine learning: An interview study. arXiv preprint arXiv:2209.09125. arxiv.org/abs/2209.09125

29. Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., & Gebru, T. (2019). Model cards for model reporting. Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT* '19), 220–229. 10.1145/3287560.3287596

30. Gebru, T., Morgenstern, J., Vecchione, B., Vaughan, J. W., Wallach, H., Daumé III, H., & Crawford, K. (2021). Datasheets for datasets. Communications of the ACM, 64(12), 86–92. 10.1145/3458723

How to cite this article

Lee, Q. F. (2026). Designing Machine Learning Systems: From Problem Framing to Production Operations. *Chmltech Online Journal*, *104*(6). https://www.chmltech.com/post/designing-machine-learning-systems-from-problem-framing-to-production

Ready to find your next improvement?

Book a strategy call. We will talk through your goals, pain points and where AI, systems or process work can make the biggest difference.

Book a Strategy Call

Or email **info@chmltech.com**